

PDD - Documento de Projeto

Chat Claude com Auditoria GPT

Versão 1.0 - 10/06/2026

1. Visão Geral

Aplicação web de chat no estilo Claude.ai com um diferencial: cada resposta da IA principal é **auditada automaticamente por uma segunda IA** - e, opcionalmente, as duas **debatem em rodadas** até a resposta convergir. O usuário conversa com o Claude (Anthropic) e o GPT (OpenAI) avalia a qualidade: confirma quando está correta ou apresenta correções quando identifica problemas. No modo de rodadas (4x a 10x), a crítica do GPT volta ao Claude, que a julga e refina a resposta, e o ciclo se repete - sempre terminando com o veredito do GPT.

O sistema roda 100% em Docker: três containers (frontend, API e Redis) orquestrados por Docker Compose. Para rodar em qualquer máquina basta ter Docker instalado, copiar o projeto, preencher o arquivo .env com as chaves de API e executar `docker compose up --build`.

2. Objetivos

- Oferecer uma interface de chat moderna, com streaming de resposta em tempo real.
- Aumentar a confiabilidade das respostas por meio de auditoria automática cruzada entre modelos de fornecedores diferentes.
- Suportar arquivos como contexto: imagens (visão computacional), PDF, TXT, MD, DOCX e XLSX.
- Permitir exportar respostas em TXT, MD e PDF.
- Manter histórico de conversas persistente entre sessões e reinícios.
- Ser totalmente portátil: nenhuma dependência instalada na máquina além do Docker.

3. Arquitetura

Três serviços orquestrados pelo Docker Compose:

- **web** (porta 8100): React 18 compilado pelo Vite e servido por Nginx. Interface em duas colunas - chat principal à esquerda, painel de auditoria à direita (vira abas no mobile), e sidebar com o histórico de conversas.
- **api** (porta 8101): FastAPI (Python 3.12). Orquestra as chamadas às IAs, processa uploads, gera downloads e persiste o histórico.
- **redis** (sem porta exposta): Redis 7 com AOF ligado. Guarda as conversas em volume Docker.

Fluxo de uma mensagem:

1. O browser envia a mensagem (e IDs de arquivos anexados) para POST /api/chat.
2. A API monta o contexto: imagens viram blocos de visão base64; documentos têm o texto extraído injetado no prompt.
3. A API chama o Claude Haiku com streaming e repassa cada chunk ao browser via SSE.
4. Ao terminar, a API chama o GPT-4o-mini com a pergunta e a resposta completa.
5. O veredito da auditoria é enviado como evento final do stream e o turno é salvo no Redis.

4. Modelos de IA

- **Resposta principal (Anthropic):** modelo selecionável na interface. A lista é obtida dinamicamente da Models API da Anthropic - aparecem todos os modelos que a chave do usuário pode usar (padrão: claude-haiku-4-5). Streaming via SDK oficial anthropic.
- **Auditoria (OpenAI):** modelo selecionável, lista dinâmica da Models API da OpenAI filtrada para modelos de chat - inclui as famílias GPT-4, GPT-4o, GPT-4.1 e GPT-5+ disponíveis para a chave (padrão: gpt-4o-mini). JSON garantido via response_format. Retorna {status: OK | REVIEW, comment, response?, via}.
- **Contexto do auditor:** o auditor (e o booster) recebe a pergunta com o conteúdo dos documentos anexados e os turnos recentes da conversa - para verificar fatos e julgar perguntas que dependem de contexto anterior sem falsos negativos. Sabe que código Python executável é uma entrega válida de arquivo e não penaliza descrições de imagens que só o modelo principal enxerga. Em conversas muito longas, o histórico enviado aos modelos é podado (as últimas ~40 mensagens), embora a conversa completa permaneça no Redis.
- **Auditor que corrige arquivos:** quando o código auditado tem erros (dados errados, bugs), o auditor entrega no painel a resposta com o código Python completo e corrigido - e o painel de auditoria também tem o botão "Executar código", entregando o arquivo correto ao usuário. Validado em simulações: tabuada errada em XLSX e fatos errados em PDF foram detectados, corrigidos e os arquivos corretos gerados.
- **Métricas por resposta:** cada resposta exibe o modelo real reportado pela API do provedor (prova de que o modelo selecionado foi o usado), tokens de entrada/saída, tempo total e até o primeiro token, velocidade (tokens/s) e custo estimado pela tabela pública de preços. O card de auditoria mostra o modelo auditor e seus tokens. Tudo persistido junto com a conversa.
- **Modo de rodadas (2x a 10x):** seletor na interface. Em 2x, fluxo padrão (resposta + auditoria). Em 4x ou mais, debate: a crítica do GPT volta ao Claude com instrução de julgá-la com rigor (a crítica pode estar errada) e reescrever a resposta completa; o GPT julga novamente, sempre dando a palavra final. O painel de auditoria mostra a linha do tempo do debate (uma crítica por rodada) e o chat exibe apenas a resposta final refinada. Se o GPT aprovar numa rodada intermediária, o debate encerra antecipadamente (economia de tokens). As métricas somam todas as rodadas.
- **Booster (opcional):** com o toggle ativado, após o veredito final os melhores modelos de cada provedor (ex.: Claude Opus e o GPT-5.x mais recente da chave) dão, em paralelo, uma opinião final independente sobre a resposta - exibida em cards destacados no painel de auditoria.
- **Failover bidirecional:** se a API da Anthropic falhar (cota, chave, instabilidade), o usuário é avisado com a mensagem exata do problema e o modelo da OpenAI assume a resposta principal em streaming. Se a OpenAI falhar na auditoria, o Claude assume a auditoria. Se ambas falharem, o erro detalha as duas causas.

- **Geração de arquivos (automática):** quando o usuário pede um arquivo (XLSX, DOCX, PDF, CSV, gráfico), o modelo escreve internamente um script Python marcado com `# gerar-arquivo`; o backend o executa automaticamente num ambiente isolado (timeout 30s, com auto-correção em caso de falha) e a resposta exhibe diretamente os cards dos arquivos prontos para download - o código fica recolhido num "ver código". Código só aparece aberto quando o usuário pede explicitamente um código/script. Vários arquivos podem ser gerados num mesmo pedido. Bibliotecas: `openpyxl`, `python-docx`, `reportlab`, `pdfplumber`, `pandas`, `matplotlib`.

5. API (contratos)

- `POST /api/chat` - corpo: `{messages, files, conversation_id, main_model, auditor_model, rounds, booster}`. Resposta: stream SSE com eventos meta (id da conversa), chunk (texto), debate (crítica intermediária por rodada), revision_start (início de resposta refinada), model_info (métricas somadas), artifacts (arquivos gerados automaticamente), booster (opiniões finais dos melhores modelos), failover, error, `[AUDIT]{json}` (veredito final com rodada e parada antecipada) e `[DONE]`.
- `GET /api/models` - modelos disponíveis para resposta principal e auditoria, com os padrões.
- `POST /api/execute` - corpo: `{code, auto_fix}`. Executa código Python isolado e retorna `{ok, stdout, stderr, files, auto_fixed?}`. Com `auto_fix` (padrão), se o código falhar o Claude o corrige com base no erro e re-executa uma vez.
- `GET /api/files/{file_id}` - download de artefatos gerados pela execução de código.
- `POST /api/upload` - multipart. Valida tamanho (padrão 10MB) e tipo. Retorna `{file_id, name, type, preview}`. Arquivos expiram por TTL (padrão 60 min).
- `POST /api/download/{txt|md|pdf}` - corpo: `{content}`. Retorna o arquivo gerado (PDF via `reportlab`).
- `GET /api/conversations` - lista resumida, ordenada por atividade.
- `GET /api/conversations/{id}` - conversa completa (mensagens + auditorias).
- `DELETE /api/conversations/{id}` - remove a conversa.
- `GET /api/health` - usado pelo healthcheck do Docker.

6. Dados e Persistência

- **Conversas:** Redis. Cada conversa é um JSON (`conv:{uuid}`) com título, mensagens e auditorias; um índice ordenado (`conversations`) permite listagem por atividade. Durabilidade via AOF no volume `redis-data`.
- **Uploads:** volume `uploads-data` montado em `/tmp/uploads` no container da API. Cada arquivo gera os bytes originais mais um JSON de metadados (texto extraído ou base64). Limpeza automática por TTL.
- A persistência é *best-effort*: se o Redis cair, o chat continua funcionando, apenas sem histórico.

7. Configuração

Tudo via arquivo `.env` na raiz (modelo em `.env.example`):

- ANTHROPIC_API_KEY - chave da Anthropic (começa com sk-ant-).
- OPENAI_API_KEY - chave da OpenAI (começa com sk-).
- MAX_FILE_SIZE_MB - limite de upload (padrão 10).
- UPLOAD_TTL_MINUTES - validade dos uploads (padrão 60).
- WEB_PORT / API_PORT - portas no host (padrão 8100/8101, faixa escolhida para não conflitar com outros projetos).
- VITE_API_URL - URL da API vista pelo browser; é embutida no build do frontend (mudou? rode `docker compose build web`).

8. Como Rodar em Qualquer Máquina

1. Instale o Docker (Docker Desktop no Mac/Windows; docker + compose plugin no Linux - Compose 2.24+).
2. Copie a pasta do projeto.
3. `cp .env.example .env` e preencha as duas chaves de API.
4. `docker compose up --build -d`
5. Abra `http://localhost:8100`.

Portabilidade verificada: o compose não usa nenhum caminho da máquina (só volumes nomeados do Docker), as imagens são construídas do zero a partir do código com dependências travadas (`package-lock.json`/versões fixadas no pip), e o teste de "máquina nova" (apagar volumes e imagens e subir com um único comando) é executado a cada rodada de mudanças. Sem o `.env`, os serviços sobem normalmente e o chat exibe um erro claro de autenticação até as chaves serem preenchidas.

Comandos úteis: `docker compose logs -f api` (logs), `docker compose down` (parar mantendo dados), `docker compose down -v` (parar apagando histórico e uploads).

9. Qualidade, Segurança e Confiabilidade

- Healthchecks nos três containers; o web só sobe depois da API saudável, e a API depois do Redis.
- `restart: unless-stopped` em todos os serviços.
- Build do frontend reproduzível com `package-lock.json + npm ci`; dependências Python com versões fixadas.
- **Segurança de uploads:** whitelist de tipos (imagens, PDF, TXT, MD, DOCX, XLSX, CSV, JSON) com validação do conteúdo real por assinatura (magic bytes) - um executável renomeado para `.png`, um binário disfarçado de `.txt` ou um falso `.docx` são rejeitados com 415, mesmo com a extensão "certa". Arquivos corrompidos que falham na extração também são rejeitados.
- **Execução de código isolada:** subprocess com `python -I` (modo isolado), diretório temporário próprio, timeout de 30s e limite de 25MB por artefato. Auto-correção opcional (1 tentativa) quando o código falha.
- Validações: 400 (requisição inválida/arquivo vazio), 413 (arquivo grande), 415 (tipo não suportado/disfarçado), 404 (conversa/artefato inexistente).

- Erros das IAs chegam ao usuário como evento SSE limpo, sem derrubar o stream; persistência e auditoria são best-effort e nunca bloqueiam o chat.

10. Testes Realizados (10/06/2026)

- Build do zero (simulação de máquina nova com volumes e imagens apagados) - os três containers sobem saudáveis com um único comando.
- Chat real com streaming e auditoria (chaves reais): resposta em chunks e veredito do GPT.
- Visão: imagem PNG vermelha identificada corretamente pelo Claude.
- Upload dos seis tipos suportados com extração de texto validada; caminhos de erro 400/413/415 confirmados.
- Download nos três formatos; PDF validado como documento real.
- Persistência: histórico sobrevive a `docker compose restart`; multi-turno com memória da conversa.

11. Limitações Conhecidas e Evolução

- Imagens anexadas em turnos anteriores não são reenviadas ao Claude nos turnos seguintes (só as da mensagem atual). No failover para a OpenAI, imagens da mensagem atual são descartadas (formatos de visão incompatíveis); o texto extraído de documentos é mantido.
- A execução de código roda dentro do container da API com timeout de 30s - adequada para uso local; para expor o app na internet seria necessário sandbox mais forte e autenticação.
- O conversor markdown para PDF cobre títulos, parágrafos justificados, tabelas estilizadas (cabeçalho repetido entre páginas), listas, citações, blocos de código com quebra de linha e rodapé com numeração; emojis são removidos (as fontes padrão de PDF não os suportam).
- O auditor ocasionalmente marca REVIEW em respostas corretas - ajustável pelo prompt em `backend/services/gpt.py`.
- Sem autenticação (uso local/rede confiável, por design).
- Evoluções possíveis: busca no histórico, instalação dinâmica de pacotes na execução, renomear conversas, autenticação para uso em rede.